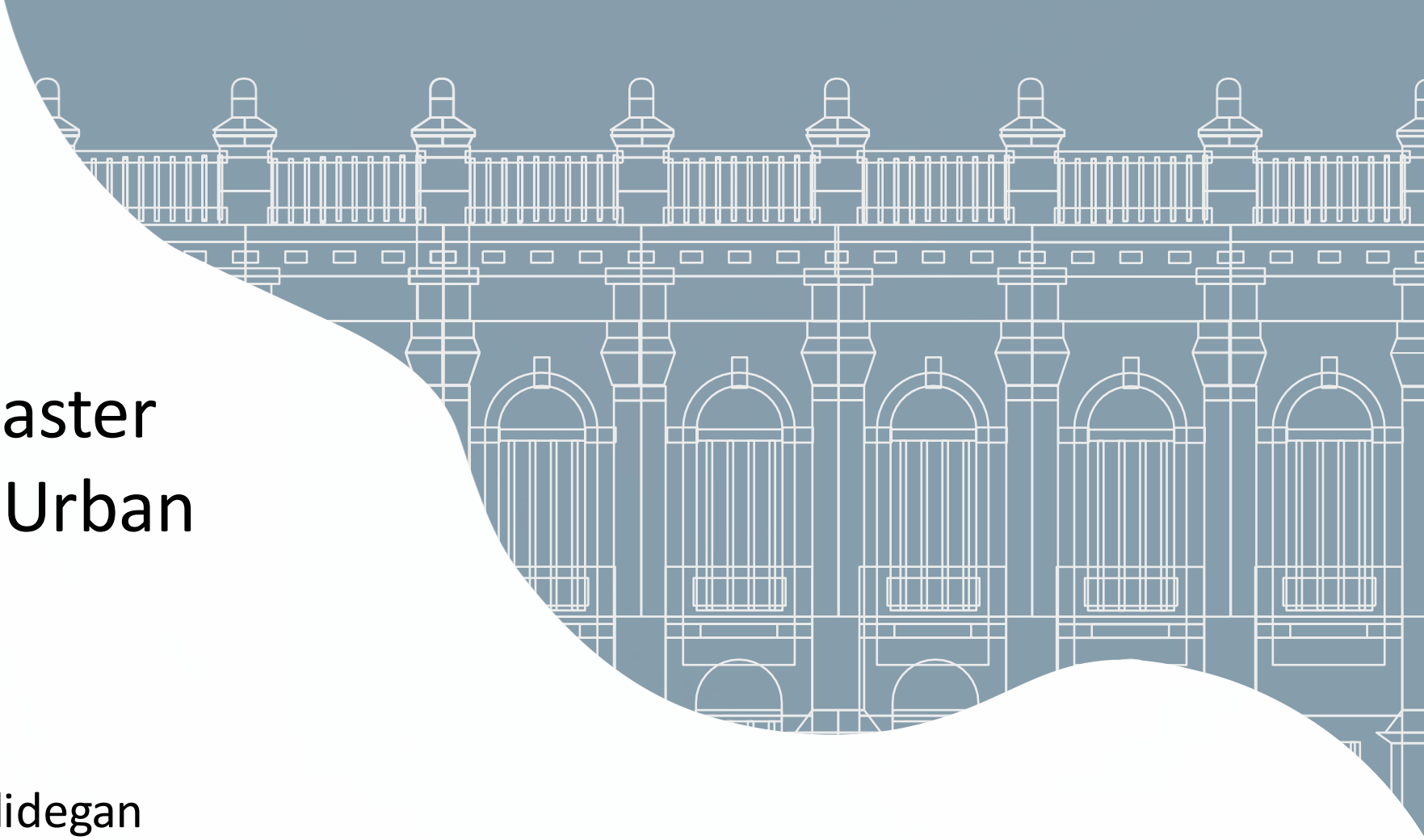
POLITECNICO
MILANO 1863

DIPARTIMENTO DI
INGEGNERIA CIVILE E AMBIENTALE



# **LayerAlterator:** A Raster Processing Tool for Urban Change Simulation

**Author:** Amirhossein Donyadidegan
**Supervisor:** Dr. Daniele Oxoli
**Course:** GeoInformatics Project (GIP), AY 2024–2025
**Date:** Jun 2025

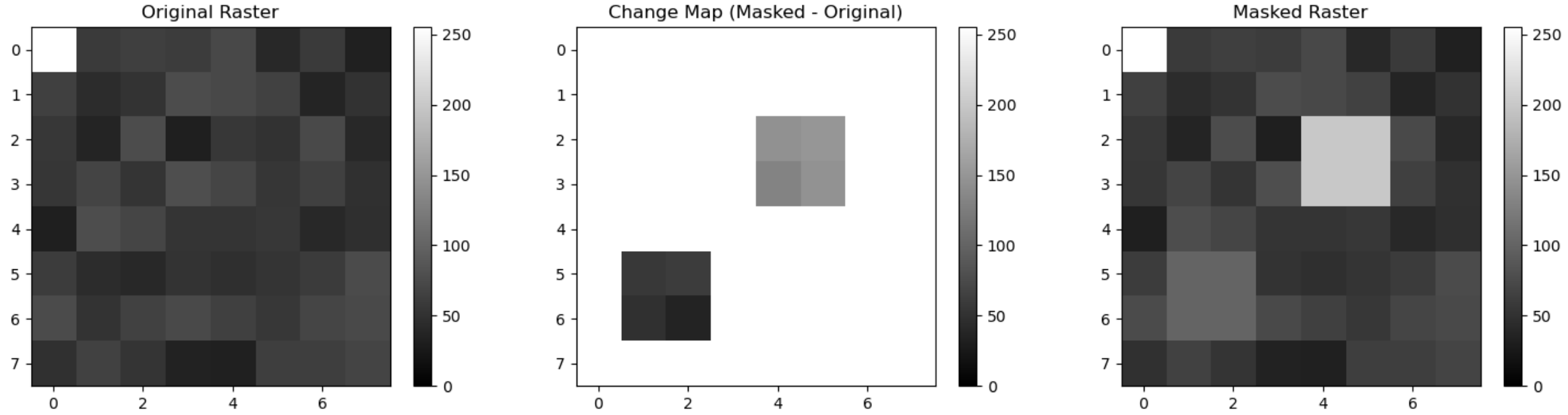
# Introduction

**Urban Change Modeling: A Need for Lightweight Tools**

- Urban planners need to test interventions like reforestation or densification.
- Existing tools (e.g. LCZ classification, full climate models) are often too heavy or rigid.

**LayerAlterator** offers a modular, fast alternative:

- Modify raster layers using **polygon-based rules**
- Supports **masking** (absolute) and **percentage** (relative) updates
- Built with open-source Python libraries



# Related Work

## Urban Raster Processing in Literature

**Local Climate Zones (LCZ)**

- Stewart & Oke (2012): Classification for urban temperature modeling
- *Rigid categories, not designed for dynamic simulation*

**Urban Land Cover Mapping**

- Okujeni et al. (2016): Fractional abundance via spectral unmixing
- *Provides detailed input layers but no simulation capability*

**Climate Modeling Platforms**

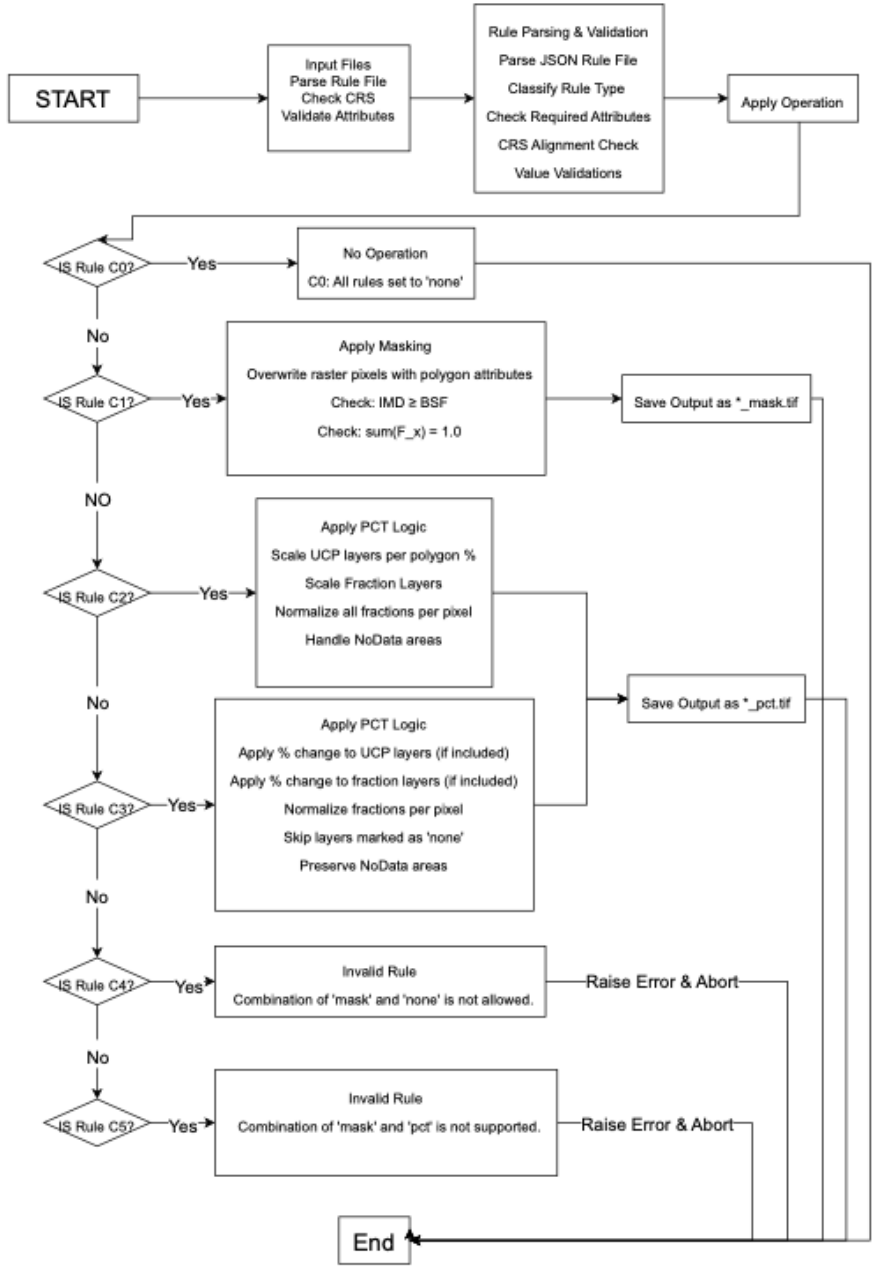
- Require high computational resources and complex inputs

| Tool | Focus | Limitation |
|---|---|---|
| LCZ | Urban class mapping | No layer modification |
| Okujeni | Fractional land cover | Static data |
| LayerAlterator | Rule-based simulation | Lightweight |

# Innovation

## What Makes LayerAlterator Unique?

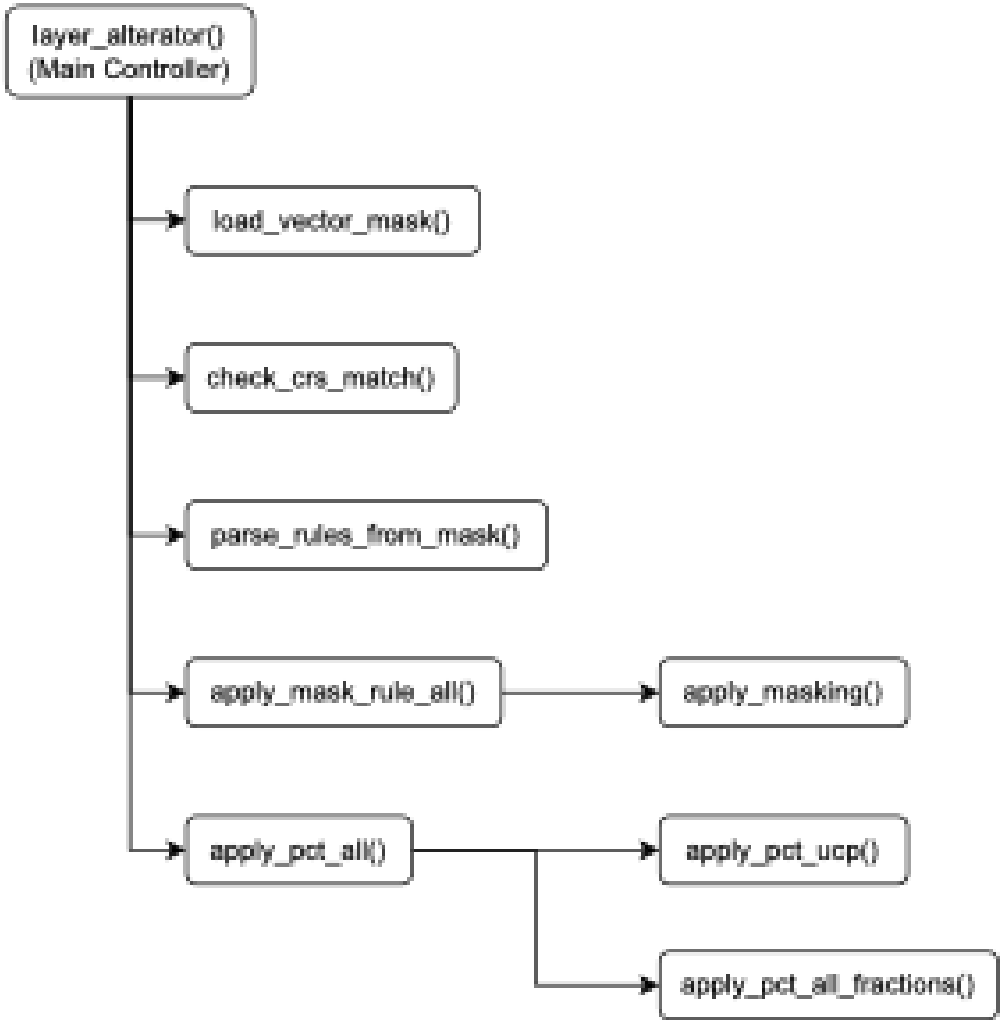
- **Supports both masking and percentage operations**
  Apply absolute values *or* modify relative to the current raster data.
- **Rule-driven engine**
  Uses simple JSON configuration for flexible simulations.
- **Modular Python architecture**
  Easy to extend, debug, or integrate into Digital Twin platforms.
- **Built-in validation**
  CRS, normalization, and attribute checks prevent faulty simulations.
- **Transparent and reproducible**
  Designed for Jupyter workflows, with visual inspection support.



# Architecture Overview

## Modular & Reproducible Processing Pipeline

- **Main Controller:** layer_alterator() orchestrates the workflow
- **Key Phases:**
  - ❑ Input Loading: load_vector_mask(), check_crs_match()
  - ❑ Rule Handling: parse_rules_from_mask(),
  - ❑ Masking Ops: apply_masking(), apply_mask_rule_all(),
  - ❑ Percentage Ops: apply_pct_ucp(), apply_pct_all_fractions(), apply_pct_all()
- Functions are independently testable and integrated in Jupyter
- Validation embedded in every major step



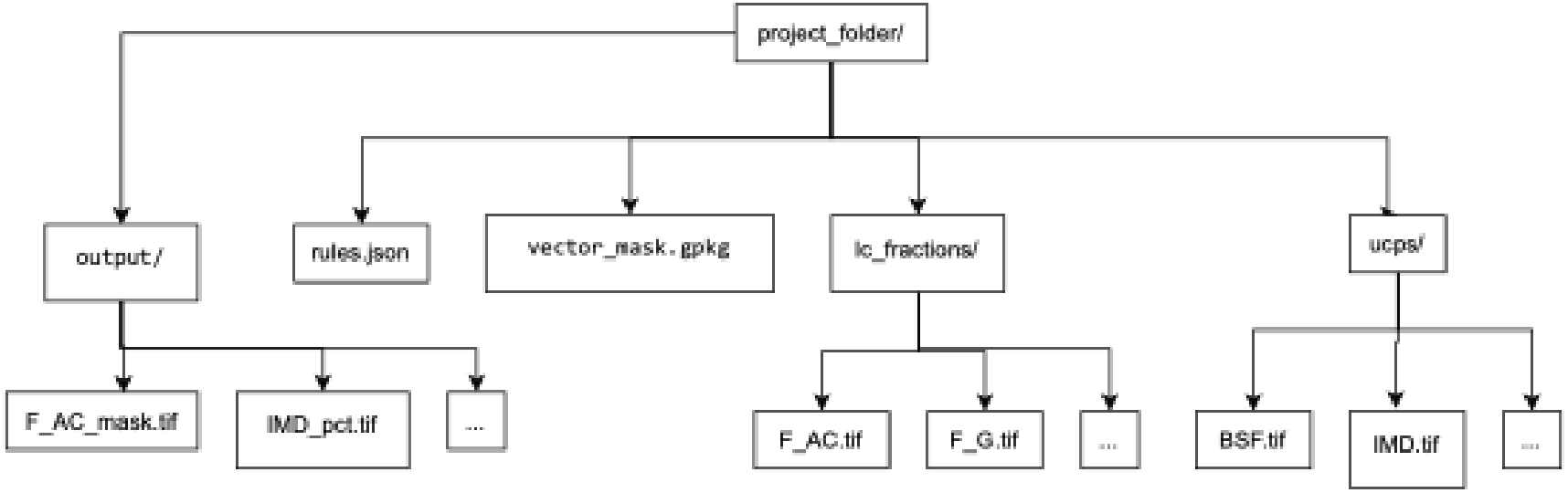
# Input and Rule Logic

## Input Types

- **Vector Mask**: Polygons (e.g., .gpkg, .geojson) with attribute values
- **Raster Layers**:
- **Rule File**: rules.json defines layer operations

## Rule Logic

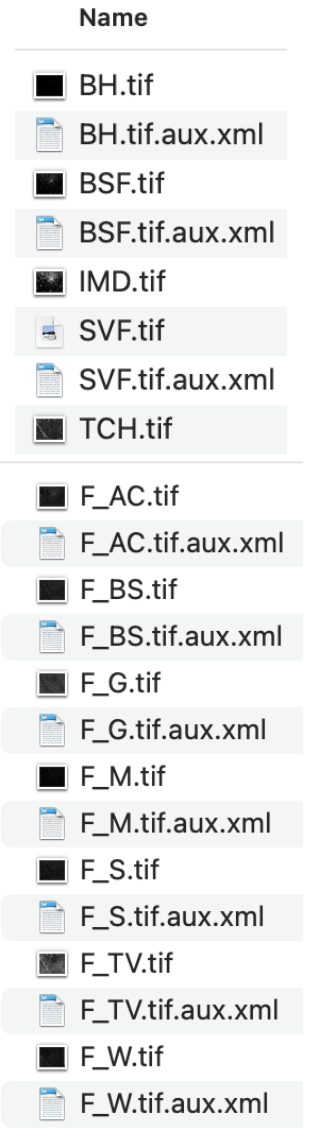
- Operations: mask, pct, none
- Classified into rule types (C0 to C5)
- Invalid mixes (e.g., mask + pct) trigger errors

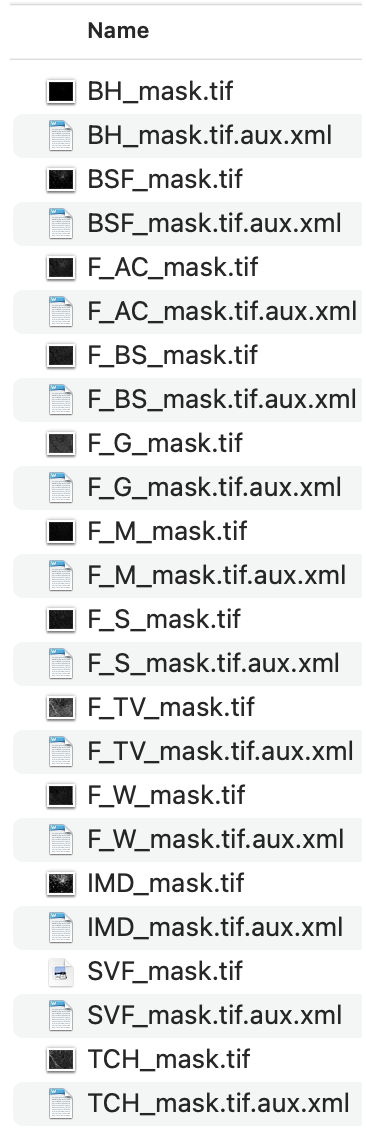


# Output Example

## Traceable Raster Output

- **Processed rasters saved in the output/ folder**
- **Naming Convention:**
  - ❑ *_mask.tif → layers updated with fixed values (masking)
  - ❑ *_pct.tif → layers updated with percentage changes
- **CRS, NoData, and resolution preserved**
- **Visual verification in QGIS (e.g., coverage, alignment, NoData integrity)**
- **Metadata and filenames support scenario comparison**



# Validation and Testing

## Ensuring Data Integrity and Simulation Accuracy

- **Validation Checks (Pre-Processing)**
  - ❑ CRS match between vector and rasters
  - ❑ Required attributes present
  - ❑ Rule logic consistency

- **During Processing**
  - ❑ NoData regions preserved
  - ❑ Normalization of fractional rasters
  - ❑ Warnings for overlapping polygons or abnormal updates

- **Testing Strategy**
  - ❑ Unit tests in Jupyter Notebook (test_functions.ipynb)
  - ❑ Visual inspection in QGIS
  - ❑ Controlled edge-case scenarios

```
--- Test 1: All CRS Match ---
All raster layers have the same CRS as the vector mask.

--- Test 2: All Rasters Match CRS but Different from Vector ---
Warning: CRS mismatch detected in the following layers:
 - BH.tif: CRS = EPSG:32632, differs from vector CRS = EPSG:7415
 - F_AC.tif: CRS = EPSG:32632, differs from vector CRS = EPSG:7415

--- Test 3: CRS Mismatch ---
Warning: Could not open raster for layer IMD_differentCRS.tif: test_data/fun_2/
    IMD_differentCRS.tif: No such file or directory
All raster layers have the same CRS as the vector mask.

--- Test 4: Missing Raster File ---
Warning: Could not open raster for layer nonexistent.tif: test_data/fun_2/nonex
    No such file or directory
All raster layers have the same CRS as the vector mask.
```

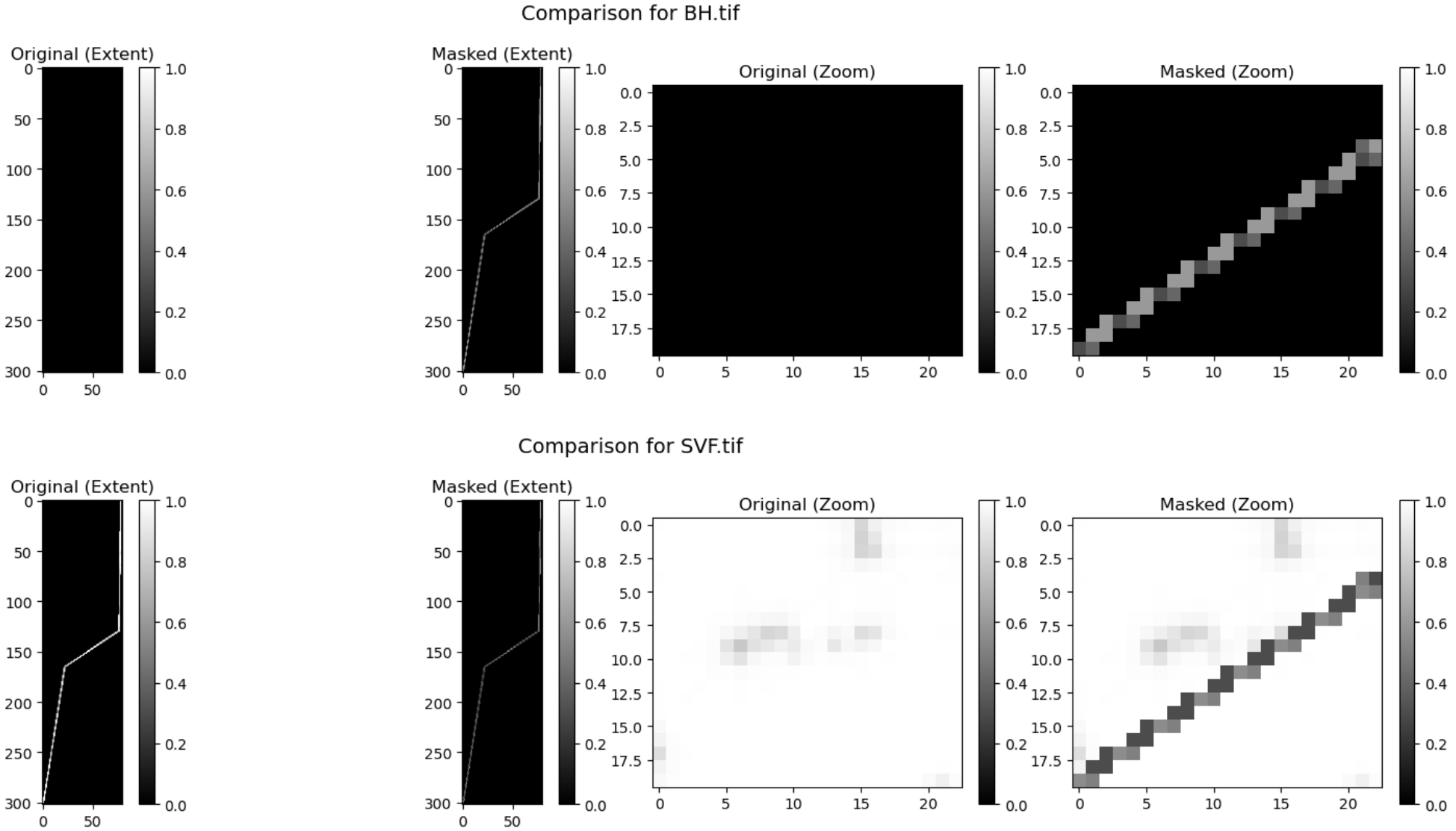
```
--- Test 4: Mix of pct and none ---
Rule C3: Layers set to PCT and NONE. Proceeding wi

--- Test 5: Mix of mask and none  ---
Rule C4 violation: MASKING + NONE is not allowed.

--- Test 6: Mix of mask and pct ---
Rule C5 violation: MASKING + PCT is not allowed.
```

# Demonstration

## Scenario: New Highway in Milan

- **Objective:** Simulate the effect of a new highway on urban surface characteristics.
- **Inputs:**
  - ❑ Vector polygon for highway area (in .gpkg)
  - ❑ Rule: Apply mask to set land cover fractions (e.g., increase F_AC, reduce F_G)
- **Execution:**
  - ❑ layer_alterator() applied with rules.json and polygon attributes
  - ❑ Outputs generated: F_AC_mask.tif, F_G_mask.tif, etc.
- **Results:**
  - ❑ Clear spatial update of asphalt fraction (F_AC)
  - ❑ Preservation of surrounding NoData
  - ❑ Ready-to-use outputs for further modeling (e.g., UHI impact)


# Demonstration



# Conclusions

## Project Achievements

- Delivered a working tool for spatial raster updates via vector-defined rules
- Successfully supports both deterministic (mask) and relative (pct) operations
- Modular architecture with embedded validation ensures reliability
- Integrated into Jupyter for transparent, reproducible workflows

## Impact & Future Work

- Valuable tool for urban planning, especially in Digital Twin contexts
- Enables “what-if” simulations with minimal overhead
- Future extensions:
  - ❑ GUI or API interface
  - ❑ Batch rule processing
  - ❑ Tiled processing for large datasets